C PROJECT REPORT
C01020 : COMPUTER SYSTEMS PROGRAMMING
GROUP 41

Overview

libtiny3d is a lightweight software rendering library written in C. It implements essential components of a 3D rendering pipeline including canvas drawing, 3D vector/matrix math, wireframe projection, simple animation, and lighting.

Task 1: Canvas & Line Drawing

Files

canvas.c, canvas.h

Key Features

- Introduced a canvas_t structure to store grayscale pixel data.
- Enabled functions to create, clear, and draw on a float-based canvas.
- Implemented thick line rendering with anti-aliasing and polar radial lines.
- Saved rendered outputs as .pgm grayscale images.

Implementation Details

- The canvas uses a 1D float array for pixel data.
- Coordinates are handled as float values for smoother graphics.
- Line drawing uses perpendicular vectors to determine width.
- Each pixel's brightness is adjusted based on distance to the line.

Mathematical Explanations

- Drawing is based on Bresenham-inspired interpolation.
- For thickness, a perpendicular unit vector is calculated.
- Intensity is inversely proportional to distance from the ideal line pat

Design Decisions

- Grayscale simplifies early-stage rendering.
- All rendering logic is modular, allowing testing in isolation.
- draw_clock_lines() draws radial lines from a center using polar angles

Functions Used

- create_canvas(width, height)
  Allocates and returns a blank float canvas
- canvas_clear(canvas, value)
  Fills the canvas with a specified background intensity
- set_pixel_f(canvas, x, y, intensity)
  Sets a pixel at a specific coordinate after bounds check
- draw_line_f(canvas, x0, y0, x1, y1, thickness)
  Draws a thick, anti-aliased line using float coordinates

- draw_clock_lines(canvas, radius, thickness)
  Draws radial lines outward from the center like a clock face
- save_canvas_as_pgm(canvas, filename)
  Saves the canvas as a .pgm grayscale image file
- free_canvas(canvas)
  Deallocates memory used by the canvas

Output

- clock_face_static.pgm: a static radial line pattern resembling a clock.

Task 2: 3D Math Foundation

Files
math3d.c, math3d.h, test_math.c

Key Features
- Defined vec3_t and mat4_t to handle 3D vectors and matrices.
- Supported vector arithmetic, dot/cross products, normalization.
- Built rotation, translation, scaling, and projection matrices.
- Provided matrix-matrix and matrix-vector multiplication.

Implementation Details
- Used column-major matrix storage for OpenGL-style math.
- Separated rotation matrices for X, Y, Z and XYZ combinations.
- Flattened index access speeds up internal operations.

Mathematical Explanations
- Rotation matrices use standard 3D formulas:
  - X-axis:

$$\begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & \cos\theta & -\sin\theta & 0 \\ 0 & \sin\theta & \cos\theta & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

- Perspective projection matrix:

$$P = \begin{bmatrix} \frac{1}{\text{aspect}\cdot\tan(\theta/2)} & 0 & 0 & 0 \\ 0 & \frac{1}{\tan(\theta/2)} & 0 & 0 \\ 0 & 0 & \frac{f+n}{n-f} & \frac{2fn}{n-f} \\ 0 & 0 & -1 & 0 \end{bmatrix}$$

Design Decisions
- Built everything from scratch without external libraries.
- Reusable components support both renderer and animation.

Function Used
Vector (vec3_t)
- vec3_new(x, y, z) / vec3_from_cartesian()
  Creates a new vector
- vec3_add(a, b), vec3_sub(a, b)
  Adds/subtracts vectors
- vec3_mul(v, s), vec3_div(v, s)
  Scales/divides vector

- vec3_dot(a, b)
  Takes the dot product
- vec3_cross(a, b)
  Takes the cross product.
- vec3_length(v) / vec3_length_squared(v)
  Calculates the vector length
- vec3_normalize(v)
  Unit vector in same direction
- vec3_distance(a, b)
  Euclidean distance
- vec3_lerp(a, b, t)
  Linear interpolation

Matrix (mat4_t)

- mat4_identity()
  Identity matrix
- mat4_translation() / mat4_translate(x,y,z)
  Translation
- mat4_rotation_x/y/z(angle)
  Rotations around each axis
- mat4_rotate_xyz(x,y,z)
  Composite XYZ rotation
- mat4_scale() / mat4_scale_uniform(s)
  Scaling.
- mat4_perspective() / mat4_ortho()
  Projection matrices
- mat4_look_at()
  View matrix simulating camera
- mat4_mul(a, b)
  Matrix multiplication
- mat4_transform_point(m, p)
  Applies projection
- mat4_transform_vector()
  Applies matrix to vector

Output

- frame_XXXX.pgm files later converted to an mp4 file

Task 3: 3D Rendering Pipeline

Files
renderer.c, renderer.h, main_soccerball.c

Key Features
- Introduced mesh_t to represent 3D wireframes.
- Built a full soccer ball using generate_soccer_ball() function.
- Performed 3D-to-2D projection with custom MVP matrices.
- Implemented circular viewport clipping and Z-depth sorting.

Implementation Details
- Soccer ball mesh uses 60 hardcoded vertices, 90 edges.
- Projection = model × view × projection.
- Z-sorting = average Z-depth of edge endpoints.
- Clipping = point must fall inside a radius from screen center.

Mathematical Explanations
- Perspective projection and viewport mapping:
  - Normalized device coordinates (NDC) in [-1,1] are converted to screen coordinates using:

$$x_{screen} = (x_{ndc} + 1) \cdot \frac{\text{width}}{2}, \quad y_{screen} = (1 - y_{ndc}) \cdot \frac{\text{height}}{2}$$

- Clipping checks:

$$(x - cx)^2 + (y - cy)^2 \leq r^2$$

Design Decisions
- Separated logic: mesh creation, projection, clipping, and drawing.
- .pgm chosen for lightweight image export.
- Used painter's algorithm for depth sorting.

Functions Used
- generate_soccer_ball()
  Builds vertices and edges of a truncated icosahedron
- render_wireframe()
  Renders mesh edges using projection and depth sorting
- project_vertex()
  Applies model-view-projection to transform 3D → 2D.
- clip_line_to_circle()
  Clipping based on circular boundary check
- create_view_matrix()
  Camera-style matrix using eye, center, up vectors

- project_vertex()
  Transforms a 3D point through model, view, and projection matrices, mapping it to screen space
- clip_to_circular_viewport()
  Checks if a point lies inside a circular viewport
- draw_line_clipped()
  Draws a line only if at least one endpoint is within the viewport.
- create_projection_matrix():
  Generates a perspective projection matrix
- free_mesh()
  Frees vertex and edge arrays of the mesh

Output

- soccer_ball_frame_000.pgm to soccer_ball_frame_119.pgm
- soccer_ball_demo.pgm (static final frame)

Task 4: Lighting & Polish

Files
lighting.c, lighting.h, animation.c, animation.h, test_visual.c

Key Features
- **I**mplemented directional lighting with brightness scaling.
- Enabled animation using cubic Bezier curves.
- Simulated moving, shaded objects with lighting response.

Implementation Details
- Lighting intensity uses Lambertian model via dot product.
- Supports multiple hardcoded light vectors.
- Bézier curves compute object positions over time.
- main.c updates animation frame-by-frame with update(dt).

Mathematical Explanations
- Lighting uses Lambertian reflection:

$$B(t) = (1-t)^3 P_0 + 3(1-t)^2 t P_1 + 3(1-t) t^2 P_2 + t^3 P_3$$

where N is the edge direction and L is the light direction.

$$I = \max(0, \hat{N} \cdot \hat{L})$$

- Bezier curves provide smooth parametric motion from 4 control points.

Design Decisions
- Used grayscale shading to simulate brightness.
- Control points and lights are hardcoded for simplicity.
- Used grayscale color scaled by intensity to simulate brightness.

Functions Used
Lighting
- compute_lighting(edge_dir, lights, light_count)
  Returns brightness (0–1) based on edge-light alignment

Animation
- bezier(p0, p1, p2, p3, t)
  Computes a point along a Bezier curve at parameter t
- update(dt)
  Updates object positions for animation loop

Output
- Animated shapes of lines moving along with lighting effects.

Individual Contributions

Vidusini Abesekara (E/23/001)

- Task 1 – Canvas and line drawing
- Task 4 – Lighting & animated scene with Bézier movement

Thenuk Piyathilake (E/23/274)

- Task 2 – Vector/matrix math engine
- Task 3 – 3D pipeline, soccer ball generation, projection, rotation

AI Tool Usage

ChatGPT and DeepSeek were a big help throughout the project. It supported us in designing clear and reusable code for math and rendering functions like the ones used for 3D transformations and drawing on the canvas. When we ran into tricky C programming issues and ChatGPT helped figure out what was wrong and how to fix it. It also explained some of the harder math concepts, like how projection matrices work or how Bezier curves are used for animation, in a way that was easy to understand. Towards the end, ChatGPT helped write and polish this report, making sure each section was clear and well-organized.